SOFTWARE QUALITY ENGINEERING

Lab 7: Coverage Analysis

White-Box Testing with Coverage.py

Student: Faizan Ali
Roll Number: FA23-BSE-022
Course / Lab: Software Quality Engineering — Lab 7
Project: University Library Management System (`auth.py`)

Final Test Result: 32 passed in 0.43 seconds
Final Coverage: 95% (above the lab's 90% checkpoint)

Purpose

This report records the baseline code coverage, summarizes the gaps identified during initial testing, documents the additional tests, and compares the final coverage results with the baseline.

---

1. Baseline Coverage

The original seven tests passed. The first recorded coverage run showed 55% total coverage.

Measure	Baseline Value
Statements	124
Missed Statements	46
Branches	50
Partial Branches	12
Total Coverage	55%
Tests Passed	7

Baseline Missing List

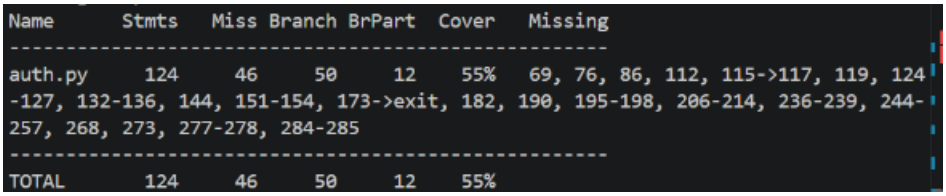
The following missing lines and branch outcomes were recorded in the terminal output:

```
69, 76, 86, 112, 115->117, 119, 124-127, 132-136, 144, 151-154, 173->exit,
182, 190, 195-198, 206-214, 236-239, 244-257, 268, 273, 277-278, 284-285
```

Interpretation

The Missing column identifies statements or branch outcomes that were not exercised by the baseline tests. These gaps indicated that the initial test suite did not cover all relevant execution paths.

The reported line numbers correspond to the version of `auth.py` used during the baseline run.



```
Name      Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------
auth.py     124     46     50     12    55%   69, 76, 86, 112, 115->117, 119, 124
-127, 132-136, 144, 151-154, 173->exit, 182, 190, 195-198, 206-214, 236-239, 244-
257, 268, 273, 277-278, 284-285
-----------------------------------------------------
TOTAL       124     46     50     12    55%
```

---

2. Gap Analysis

The baseline results indicated that additional tests were needed for alternative execution paths and error conditions, rather than testing successful login scenarios alone.

The following table maps the reported coverage gaps to suggested test scenarios.

Baseline Gap / Area	Condition Not Covered	Test Idea and Expected Result	Priority
69, 76, 86, 112	Alternate login and lockout outcomes	Test invalid input, unknown users, locked accounts, and failed-login thresholds. Verify the expected error and account lock status.	High
115->117, 119, 124-127	Login and lockout branch outcomes	Exercise valid login and boundary conditions around failed attempts. Verify authentication results and account state.	High
132-136, 144, 151-154	Account helper branches	Test locking, unlocking, unknown-account handling, and resetting failed attempts. Verify stored flags and counters.	Medium
173->exit, 182, 190	Session creation and validation paths	Test missing, valid, and expired sessions. Verify the result and ensure expired sessions are removed where appropriate.	High
195-198, 206-214	Reset-token marking and verification	Test valid, unknown, expired, and used tokens. Expect `True` only for a valid, unused token.	High
236-239, 244-257	Reset-email outcomes	Test unknown email addresses, rate limiting, token-generation exceptions, and SMTP `ConnectionError`. Verify the failure result and corresponding error.	High

268, 273, 277-278, 284-285	Remaining helper and error branches	Add targeted tests for the relevant helper paths identified in the annotated HTML report. Verify specific return values or state changes.	Medium

Verifying the Exact Line-to-Function Mapping

Open `htmlcov/index.html` in a browser and select `auth.py`. Inspect the highlighted lines to identify the uncovered statements and branches.

Terminal output identifies missing line numbers, while the annotated HTML report provides more context about the source code. Function names should be confirmed from the actual source rather than inferred from line numbers alone.

---

3. Tests Added and Final Results

Additional tests targeted reset-token validation, account lockout behaviour, login rejection paths, session handling, account helper functions, and reset-email failure scenarios.

The recorded final test run contained 32 passing tests.

Baseline vs. Final Coverage

Measure	Baseline	Final	Change
Tests Passed	7	32	+25 passing tests
Statement Count	124	124	No change
Missed Statements	46	2	44 fewer missed statements
Branches	50	50	No change
Partial Branches	12	7	5 fewer partial branches
Total Coverage	55%	95%	+40 percentage points

Final Coverage Report

The recorded final terminal report showed:

- **File:** `auth.py`
- **Total Statements:** 124
- **Missed Statements:** 2
- **Branches:** 50
- **Partial Branches:** 7
- **Total Coverage:** 95%

Final Missing List

The final report recorded the following missing lines and branch outcomes:

```
126->exit, 134->exit, 153->exit, 173->exit, 197->exit, 213-214, 238->exit
```

These entries identify remaining statements or branch outcomes that were not fully covered by the tests.

Coverage Limitation

Code coverage indicates which statements and branch outcomes were executed; it does not independently prove that the tests verified the correct behaviour.

A test can increase coverage without meaningfully checking the application's behaviour. Therefore, each test should include appropriate assertions that verify the expected return value, error, or state change.

---

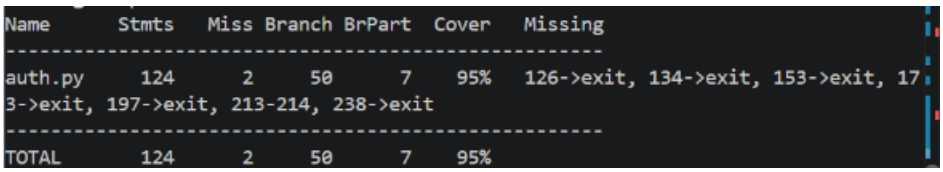
4. Residual Gaps and Suggested Tests

The final missing list contains two missed statements at lines 213-214, together with several partially covered branch outcomes.

The branch-exit entries indicate that an alternative decision outcome was not exercised. These locations should be inspected in the annotated HTML coverage report before assigning them to specific functions.

Additional tests should target the relevant alternative conditions and assert the resulting return values or state changes.

The final recorded coverage of 95% is above the lab's stated 90% checkpoint. However, the remaining gaps should still be reviewed to improve the completeness of the test suite.



Conclusion

The baseline testing phase achieved 55% code coverage with seven passing tests. After additional tests were introduced for authentication, account lockout, session management, reset-token validation, and error-handling scenarios, the recorded final coverage increased to 95%, with 32 passing tests.

The results demonstrate how white-box testing and coverage analysis can help identify untested execution paths and guide the development of additional test cases. Although the final coverage exceeds the lab's 90% checkpoint, the remaining uncovered statements and branch outcomes should be examined to further improve test completeness and reliability.